Estrategia de Branching y Control de Cambios

Type	Strategy
Created time	November 7, 2026 10:33 AM
Project	circle-guard

En este archivo se define la creación, revisión y cómo se nombran las ramas y los cambios del proyecto CircleGuard, manteniendo la trazabilidad con Jira y el flujo de CI/CD.
Estas decisiones fueron tomadas por nuestro equipo.

Daniela Castaño Moreno

Santiago Alejandro Estrada Martínez

Faiber Stiven Piedrahita Perlaza

1. Estrategia elegida: GitHub Flow

Se eligió **GitHub Flow** por la necesidad de integración y entrega continua (CI/CD) y por la facilidad de uso (hemos usado GitHub toda la carrera); revisamos documentación y vimos que GitFlow es más complejo y para el tiempo que tenemos, puede que la falta de conocimiento y su curva de aprendizaje más alta retrase nuestros entregables. Además, GitHub Flow su compatibilidad con el pipeline Multibranch de Jenkins y su relación con la gestión ágil en Jira.

Elemento	Regla documentada
`main`	Única rama de larga duración; debe mantenerse lista para desplegar, con el pipeline en verde. Está protegida.
Ramas de trabajo	Son efímeras y se crean desde `main`. Se contempla una rama por cada HU o tarea de Jira del Sprint 1.

Elemento	Regla documentada
`dev`	Se menciona en la estructura inicial del repositorio.

Por confirmar: la descripción inicial incluye `dev`, pero la regla de GitHub Flow indica que `main` es la única rama de larga duración. Definir si `dev` se conserva y cuál será su función.

2. Protección y control de cambios

Los cambios hacia `main` siguen estas reglas:

1. **Sin push directo:** los cambios se integran mediante un Pull Request (PR).
2. **Revisión por pares:** cada PR requiere al menos una aprobación de otra persona del equipo.
3. **Checks obligatorios:** el pipeline de Jenkins debe aprobar el build, las pruebas unitarias y el análisis de SonarQube antes de habilitar el merge.

3. Trazabilidad con Jira

Para conectar la gestión del trabajo en Jira con el código en GitHub:

- **Ramas:** deben comenzar con el ID de la tarea de Jira.
  - `CG-HU-<microservicio>-12-feat/implementar-login (resumen corto de la HU)`
  - `CG-<microservicio>-15-fix/error-de-token (descripción corta)`
- **Commits:** deben incluir el ID de Jira al final del mensaje.

4. Convención de commits

Se utiliza **Conventional Commits** para facilitar el versionado semántico (SemVer) y la generación automática de notas de versión.

Formato:

```
<tipo>(<alcance>): <descripción> [ID-JIRA]
```

Tipo	Uso / efecto documentado
`feat`	Nueva funcionalidad; incrementa MINOR.
`fix`	Corrección de bug; incrementa PATCH.
`feat!` o `BREAKING CHANGE`	Cambio incompatible; incrementa MAJOR.
`chore`, `ci`, `docs`, `test`, `refactor`, `build`	No incrementan la versión.

Ejemplos:

- `feat(promotion): agrega cascada de estados con TTL de 14 dias [CG-20]`
- `fix(gateway): corrige expiracion de QR firmado [CG-21]`
- `feat(auth)!: reemplaza esquema de roles [CG-22]`
- `ci(jenkins): agrega etapa de analisis con SonarQube [CG-10]`

5. Convención de nombres de ramas

La documentación registra este formato para las ramas de funcionalidad del Sprint 1:

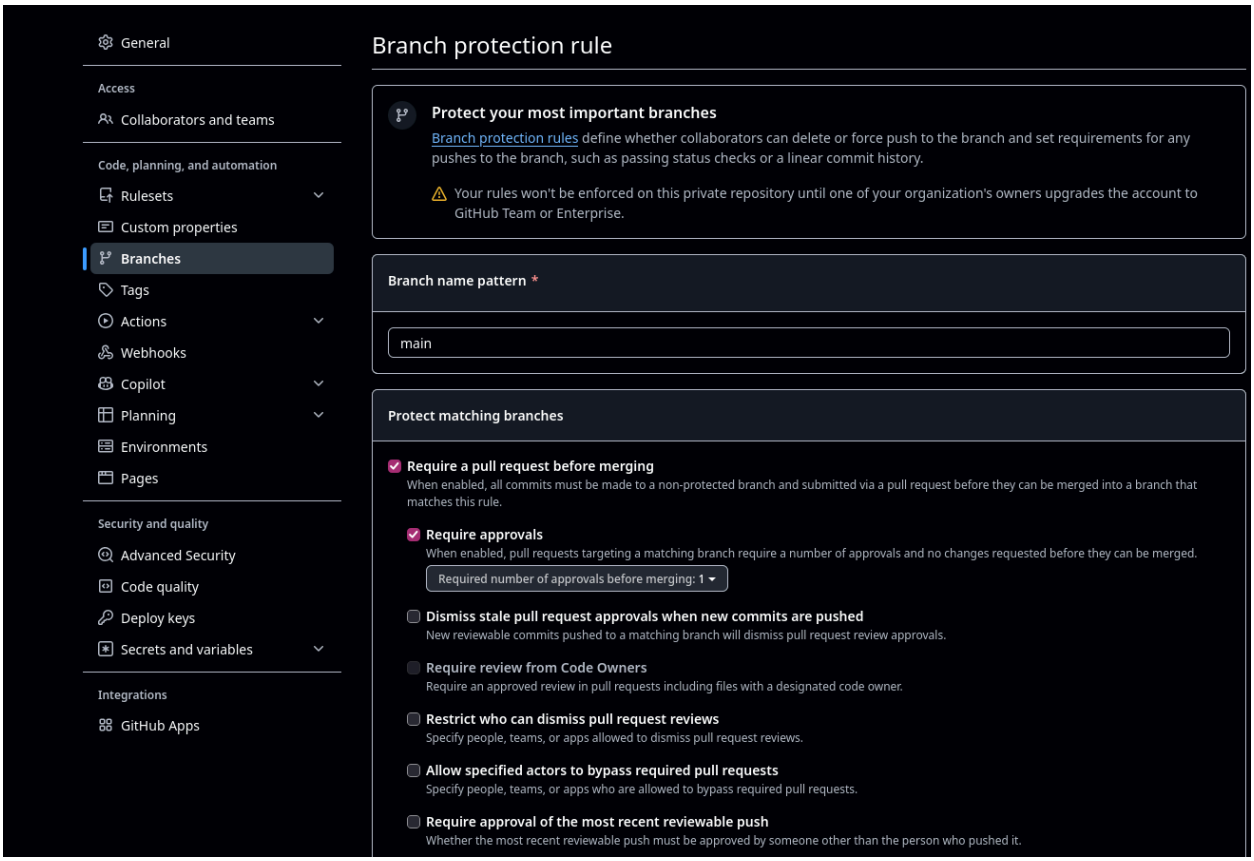
```
HU-<nombre del microservicio>-<número de HU>-<descripción corta de funcionalidad>
```

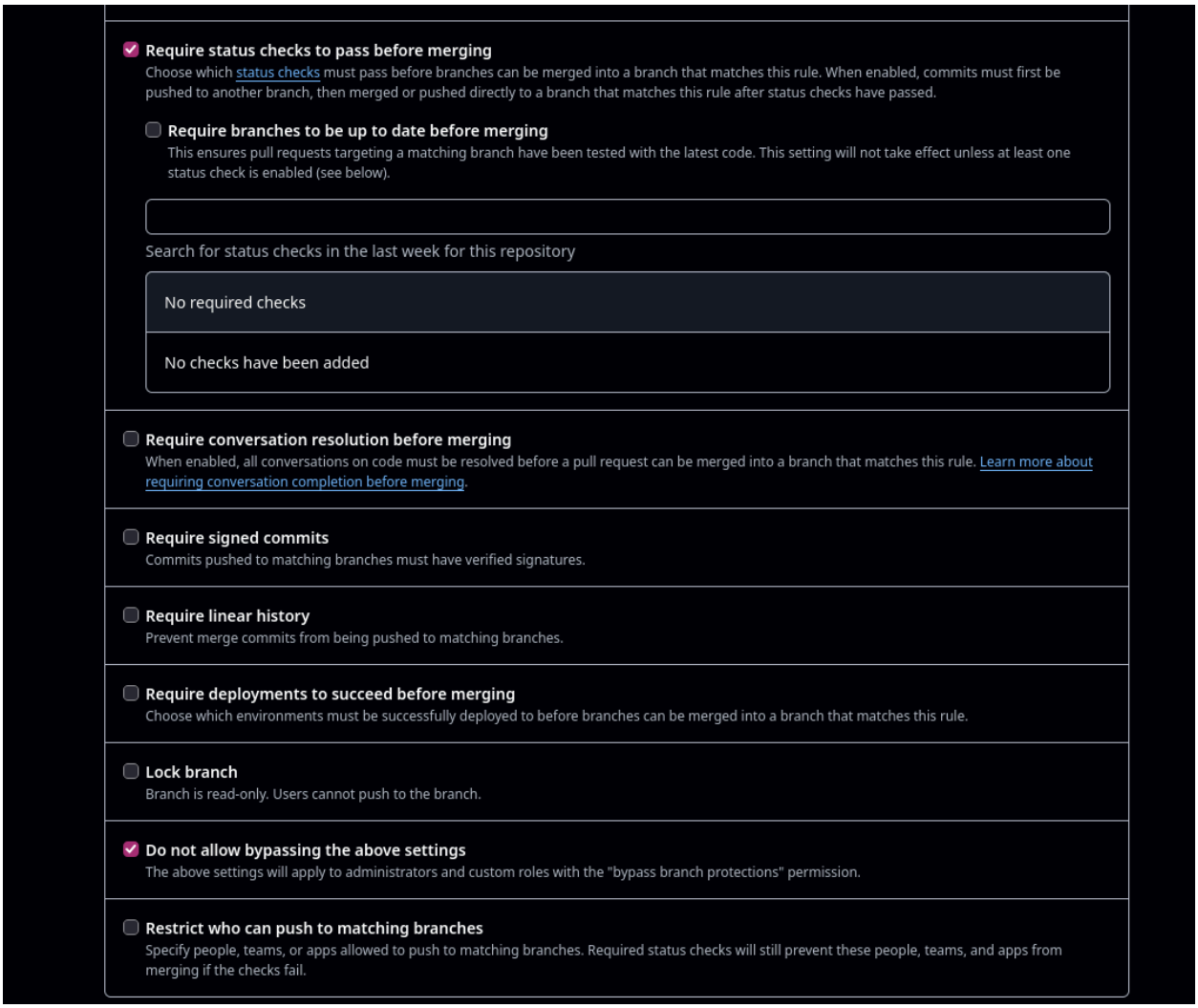
También establece que las ramas deben comenzar con el ID de Jira, como en los ejemplos de la sección anterior.

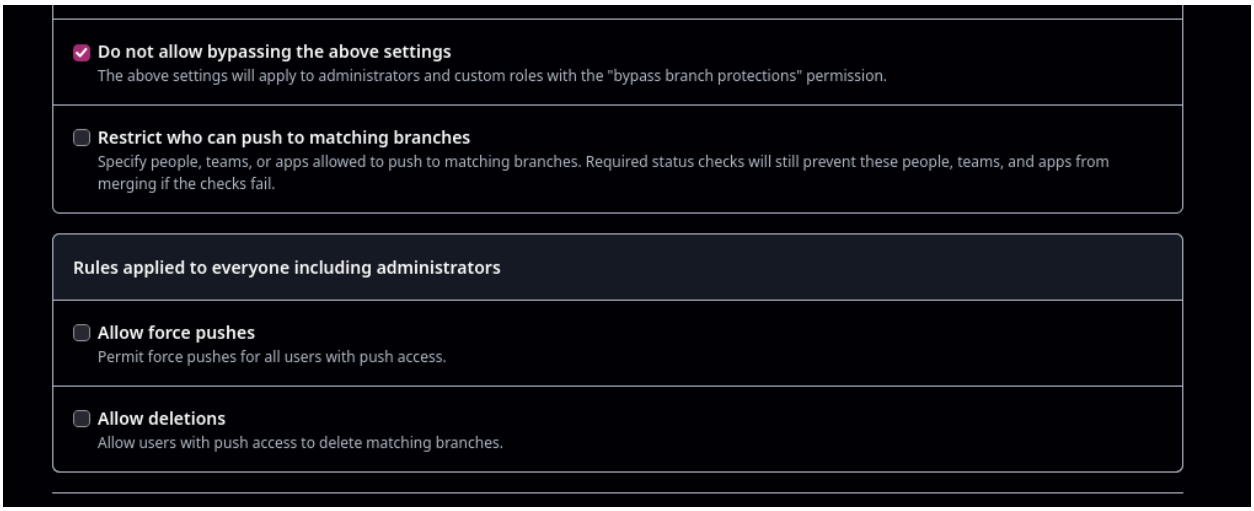
Por unificar: aparecen dos formatos de nombre de rama —`HU-...` y `CG-...`—. Acordar uno para que la convención sea consistente.

6. Capturas

🛠️ **Pendiente:** reemplazar la captura correspondiente cuando Jenkins esté disponible.


General
Access
Collaborators and teams
Code, planning, and automation
Rulesets
Custom properties
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Pages
Security and quality
Advanced Security
Code quality
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Branch protection rule
Protect your most important branches
Branch protection rules define whether collaborators can delete or force push to the branch and set requirements for any pushes to the branch, such as passing status checks or a linear commit history.
Your rules won't be enforced on this private repository until one of your organization's owners upgrades the account to GitHub Team or Enterprise.
Branch name pattern *
main
Protect matching branches
Require a pull request before merging
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
Require approvals
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.
Required number of approvals before merging: 1
Dismiss stale pull request approvals when new commits are pushed
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.
Require review from Code Owners
Require an approved review in pull requests including files with a designated code owner.
Restrict who can dismiss pull request reviews
Specify people, teams, or apps allowed to dismiss pull request reviews.
Allow specified actors to bypass required pull requests
Specify people, teams, or apps who are allowed to bypass required pull requests.
Require approval of the most recent reviewable push
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.


Require status checks to pass before merging
Choose which status checks must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.
Require branches to be up to date before merging
This ensures pull requests targeting a matching branch have been tested with the latest code. This setting will not take effect unless at least one status check is enabled (see below).
Search for status checks in the last week for this repository
No required checks
No checks have been added
Require conversation resolution before merging
When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches this rule. Learn more about requiring conversation completion before merging.
Require signed commits
Commits pushed to matching branches must have verified signatures.
Require linear history
Prevent merge commits from being pushed to matching branches.
Require deployments to succeed before merging
Choose which environments must be successfully deployed to before branches can be merged into a branch that matches this rule.
Lock branch
Branch is read-only. Users cannot push to the branch.
Do not allow bypassing the above settings
The above settings will apply to administrators and custom roles with the "bypass branch protections" permission.
Restrict who can push to matching branches
Specify people, teams, or apps allowed to push to matching branches. Required status checks will still prevent these people, teams, and apps from merging if the checks fail.


Do not allow bypassing the above settings
The above settings will apply to administrators and custom roles with the "bypass branch protections" permission.
Restrict who can push to matching branches
Specify people, teams, or apps allowed to push to matching branches. Required status checks will still prevent these people, teams, and apps from merging if the checks fail.
Rules applied to everyone including administrators
Allow force pushes
Permit force pushes for all users with push access.
Allow deletions
Allow users with push access to delete matching branches.


Branch protection rule created.

Fuentes

- GitHub Flow vs. Git Flow: What's the Difference?